

aegis-statefulset — DR Demo Report

DR demo report — staging chaos drill, 2026-05-14

Scope of this report

Evidence file provenance

§ 1 — Pod-level chaos drill

§ 2 — AZ + region-level scenarios (designed, not executed)

§ 3 — Two-tier observability verification

Reproducing this drill

Cross-references

DR demo report — staging chaos drill, 2026-05-14

Narrative wrapper for the evidence pack in this directory. Each section states a hypothesis the architecture made, the method used to test it, the observed result, and the specific evidence files that back the result. Designed so a reviewer can follow the experiment without opening every raw JSON / log.

Scope of this report

Tier	Hypothesis	Executed today?	Section
Pod-level chaos	StatefulSet controller re-creates a killed pod with EBS PVs re-attached, state preserved at the volume layer	✓ Yes	§ 1
AZ-level rotation	Master-AZ failure → scale a standby AZ's node group up (<code>desired=0</code> → <code>N</code>), Velero restore into new AZ	✗ Designed, not executed	§ 2
Region-level failover	Cold DR cluster rehydrate from cross-region S3 + EBS snapshot	✗ Designed, not executed	§ 2
Two-tier observability	Grafana IaC pipeline + AWS-native CloudWatch both reach evidence-grade	✓ Yes	§ 3

Honest scope: only § 1 ran end-to-end on the live staging cluster. § 2's two scenarios are documented in ADR-04 + docs/operations/ region-failure-recovery.md + docs/operations/dr-during-relocation.md but were not executed in this drill window — they require a destructive intent the staging-account budget does not justify before Stage 3. § 3 captures the observability slice we could verify without app instrumentation.

Evidence file provenance

Every artefact in this directory pairs with a single, reproducible production step. The table below maps each file to the exact action that produced it. Anything not in the table is derived (e.g. the PDF render of this DR_REPORT.md itself).

File	Produced by	When in the drill
chaos-stateful-pod-kill.log	scripts/chaos/demo-stateful-pod-kill.sh (tee'd to /tmp/, then copied here)	Stage 0–6 of the drill, all stages
cloudwatch/00-caller-identity.json	aws sts get-caller-identity invoked by scripts/chaos/capture-cloudwatch-evidence.sh	Identity check at start of capture
cloudwatch/ebs-data-...-Volume{Read,Write}Bytes.json	aws cloudwatch get-metric-statistics --namespace AWS/EBS --metric-name VolumeWriteBytes / VolumeReadBytes ... invoked per metric × per volume by the capture script	Post-chaos capture (window = 30 min around the chaos event)
cloudwatch/ebs-data-...-Volume{Read,Write}Ops.json	Same as above, metric VolumeReadOps / VolumeWriteOps	Same
cloudwatch/ebs-data-...-VolumeQueueLength.json	Same as above, metric VolumeQueueLength	Same
cloudwatch/ebs-data-...-VolumeIdleTime.json	Same as above, metric VolumeIdleTime	Same
cloudwatch/ebs-wal-...-Volume*.json	Identical pulls for the wal PV's EBS volume	Same
cloudwatch/dynamodb-...-Consumed{Read,Write}CapacityUnits.json	aws cloudwatch get-metric-statistics --namespace AWS/DynamoDB --metric-name Consumed{Read,Write}CapacityUnits --dimensions Name=TableName,Value=aegis-statefulset-placement-staging	Same window
cloudwatch/dynamodb-...-ProvisionedReadCapacityUnits.json	Same pattern, metric ProvisionedReadCapacityUnits	Same

File	Produced by	When in the drill
cloudwatch/dynamodb-...-SuccessfulRequestLatency.json	Same pattern, metric SuccessfulRequestLatency	Same
cloudwatch/eks-control-plane-list-AWS-EKS.json	aws cloudwatch list-metrics --namespace AWS/EKS --dimensions Name=ClusterName,Value=aegis-statefulset-staging	Shape-of-available-metrics check
cloudwatch/eks-control-plane-list-ContainerInsights.json	Same pattern, namespace ContainerInsights (returns empty — addon not enabled, documented non-result)	Same
cloudwatch/eks-audit-pod-delete.json	aws logs filter-log-events --log-group-name /aws/eks/aegis-statefulset-staging/cluster --filter-pattern '{ \$.verb = "delete" && \$.objectRef.resource = "pods" && \$.objectRef.namespace = "aegis-app" }' invoked by the capture script	Captured the two audit events from Stage 4 (operator delete + Stage 5 kubelet cleanup)
grafana-1-stack.png	Manual browser screenshot — https://aegis.grafana.net after login, stack landing page	Steady state, before chaos
grafana-2-dashboards-list.png	Manual browser screenshot — Grafana → Dashboards menu → eight aegis-statefulset dashboards listed	Same
grafana-3-service-availability-nodata.png	Manual browser screenshot — click into service-availability dashboard, panels render with No data markers	Same
cloudwatch-1-audit-pod-delete.png	Manual browser screenshot — CloudWatch Logs Insights query fields @timestamp, verb, objectRef.namespace, objectRef.name, user.username, responseStatus.code filter verb = "delete" and objectRef.resource = "pods" and objectRef.namespace = "aegis-app" over chaos window	Post-chaos
cloudwatch-2-ebs-volume-io.png	Manual browser screenshot — CloudWatch Metrics → EBS Per-Volume-with-Instance-ID → filter to both PV volume IDs → graph	Post-chaos

File	Produced by	When in the drill
cloudwatch-ebs-volume-io.json	VolumeAvgIOPS + VolumeAvgWriteLatency over chaos window Inline aws cloudwatch get-metric-statistics dump for both volume IDs over the chaos window (text-format companion to the PNG)	Same
cloudwatch/ directory wrapper	Created by the capture script; one JSON per (namespace × metric × dimension) call	Capture-script execution

Reproducer chains — what to run to regenerate each class of file:

To re-generate...	Run this	Then this
chaos-stateful-pod-kill.log	scripts/chaos/demo-stateful-pod-kill.sh	cp /tmp/chaos-stateful-pod-kill-*.log docs/evidence/chaos-stateful-pod-kill.log
All cloudwatch/*.json files	scripts/chaos/capture-cloudwatch-evidence.sh (env: AWS_PROFILE=aegis-staging-admin + optional WINDOW_START / WINDOW_END for the capture window)	(Outputs directly into docs/evidence/cloudwatch/)
Grafana / CloudWatch console PNGs	Manual browser session against aegis.grafana.net + CloudWatch console; URLs in the runbook	Save PNGs into docs/evidence/ with the canonical names

§ 1 — Pod-level chaos drill

Hypothesis

If a stateful pod is destroyed (any cause — pod-evict, node terminate, operator-triggered delete), the architecture should:

1. **Identity changes.** The StatefulSet controller creates a fresh pod object — new UID, new pod IP. (Proves the recovery is via controller reconciliation, not a kubelet restart-in-place.)
2. **State survives.** The two EBS PVs (data 18 GiB + wal 2 GiB) re-attach to the new pod by their existing PVC bindings. The PV identity is preserved — pvc-9eb95bc8... / pvc-d6639882... survives the chaos. (Proves the EBS detach → CSI attach cycle works end-to-end; the storage layer is the system of record, the pod is ephemeral.)

3. **Recovery is fast enough for the spec's RPO.** End-to-end pod- delete to pod- Ready within 5 minutes. (The spec's RPO ≤ 6 h is for data loss, not recovery time — but a pod-level chaos recovering in minutes is a precondition for the larger AZ-failure RTO budget.)

Method

scripts/chaos/demo-stateful-pod-kill.sh

Six stages, idempotent, exit-coded:

Stage	Action	Evidence captured
0	Print run metadata	log header (cluster ctx, namespace, target pod, timestamps)
1	Baseline — list pod, PVCs, bound PV identities	pod IP / node / age, PVC table, PV identities
2	Capture pre-chaos PV-identity (sidecar mounts /data read-only by design, so the durability claim is the PVC→PV binding survival, NOT a marker-file write)	OLD_DATA_PV / OLD_WAL_PV strings
3	Capture pre-chaos pod identity	OLD_UID / OLD_NODE / OLD_START
4	Chaos — <code>kubectl delete pod</code>	delete timestamp
5	Wait for new pod Ready (<code>kubectl wait --for=condition=Ready --timeout=180s</code>)	new-pod-observed timestamp, Ready timestamp
6	Verify (a) <code>OLD_UID != NEW_UID</code> — proves real recreate (b) <code>OLD_DATA_PV == NEW_DATA_PV</code> (c) <code>OLD_WAL_PV == NEW_WAL_PV</code>	UID transition + PV-identity match

Exit 0 on PASS, non-zero on FAIL with diagnostic dump of the failing pod.

Result

Captured from log `chaos-stateful-pod-kill.log` (PASS):

Metric	Value
Delete timestamp	2026-05-14T15:44:27+02:00 (= 13:44:27 UTC)
New pod observed	2026-05-14T15:45:29+02:00 (≈ 1 s after grace period)
Pod Ready	2026-05-14T15:45:32+02:00
End-to-end recovery	~ 65 s (delete → Ready)
Old UID → New UID	e1589dc3-cec3-47c4-bd1b-05593c3f8c6e → c46af96f-a98d-497a-a99b-8038b9dbeb10 (different)

Metric	Value
Old node → New node	ip-10-0-26-109 → ip-10-0-26-109 (same, master-AZ pinned — expected per ADR-01)
Data PV identity	pvc-9eb95bc8-cfdc-4b05-860d-ce0ed8a2257d (preserved)
WAL PV identity	pvc-d6639882-edda-4db6-9d49-176c4d064084 (preserved)
Pre-chaos kubectl get pods	StatefulSet 1/1 READY, pod 2/2 Running
Post-chaos kubectl get pods	StatefulSet 1/1 READY, pod 2/2 Running
terraform changes	none (entirely runtime — no IaC edit triggered)

Conclusion

All three hypotheses confirmed. Pod identity transitioned, PV identity preserved across destruction, recovery wallclock well inside the AZ- failure RTO budget (~25 min) — there's plenty of headroom for the LevelDB warm-up tail that the AZ-failure path adds (see ADR-04).

AWS-side corroboration

The K8s-side chaos log is self-evidence. Independent corroboration came from the AWS audit + metric APIs:

[cloudwatch/eks-audit-pod-delete.json](#) — two pod-delete API events both `responseStatus.code = 200`:

@timestamp (UTC)	verb	user.username	response
13:44:27	delete	.../AWSControlTowerExecution/aegis-thu-demo (the operator)	200
13:45:28	delete	system:node:ip-10-0-26-109... (kubelet finalising the pod)	200

61 s gap between the operator-trigger event and the kubelet-finalise event corroborates the 65 s in-cluster wallclock (4 s difference is the kubelet's local cleanup delta after the API call returned). Plus human-readable rendering of the same data in [cloudwatch-1-audit-pod-delete.png](#).

EBS per-volume IO metrics over the chaos window ([cloudwatch/ebs-data-...-Volume*.json](#) + the [time-series PNG](#)) — `VolumeAvgIOPS` and `VolumeAvgWriteLatency` for both PVs across the detach → re-attach window. 27 datapoints per metric × 6 metrics × 2 volumes = 324 datapoints total available for independent timing analysis.

DynamoDB placement table activity over the same window ([cloudwatch/dynamodb-...-Consumed{Read,Write}...json](#)) — consumption stays at 0, as expected. Pod-level chaos is the wrong test for the routing path (ADR-03 placement-table lookup); the table is provisioned but the drill doesn't exercise it. Documenting the non-result is the point — a drill that incidentally triggered routing-table activity would have been a configuration smell.

§ 2 — AZ + region-level scenarios (designed, not executed)

These were not run in the staging-drill window. They are documented end-to-end and the architecture is ready to run them; the staging- account budget does not justify the destructive intent before a Stage 3 conversation.

AZ-level rotation (ADR-04 + ADR-01)

- Topology: master AZ A active, AZ B + C at `desired=0` per `aws_eks_node_group.stateful_standby[*]`.
- Trigger: `aws eks update-nodegroup-config` to scale one standby's desired-size from 0 → 3.
- Velero restore into the new AZ's PVs.
- Expected RTO: ~25 min (dominated by LevelDB warm-up; documented in ADR-04 with the FSR-not-enabled caveat).
- Runbook: `docs/operations/region-failure-recovery.md` + ADR-04.

Region-level failover (ADR-04, cold-DR posture)

- Trigger: `terraform-apply` the DR-region module (no warm cluster in steady state — see ADR-10 § 7 cost dial for the "pilot light" / "warm standby" alternative ladders).
 - Velero restore from cross-region S3 BSL + EBS Snapshot copies.
 - Expected RTO: ~50 min (cluster create + node groups + Velero restore + LevelDB warm-up).
 - Reasoning: `docs/operations/why-cold-dr.md`.
 - Cost-RTO trade dial: `docs/adr/ADR-10-finops.md` § 7.
-

§ 3 — Two-tier observability verification

Grafana Cloud tier (app-layer telemetry destination)

Layer	Verified	Evidence
Stack provisioned via Terraform (<code>grafana_cloud_stack.main</code>)		<code>grafana-1-stack.png</code> shows the stack landing page on <code>aegis.grafana.net</code> , EU realm <code>prod-eu-west-4</code> .
		

Layer	Verified	Evidence
Dashboards-as-code pipeline reaches Grafana (<code>grafana_dashboard.default[*]</code> consuming JSON from <code>gitops/grafana/dashboards/</code>)		<code>grafana-2-dashboards-list.png</code> shows the eight default dashboards present.
Panel structure + PromQL queries ship via IaC	✓	<code>grafana-3-service-availability-nodata.png</code> shows the service-availability dashboard open with its full RED-method panel grid. "No data" on every panel — the in-cluster kube-prometheus-stack helm release failed install during the staging deploy (LB Controller webhook race + Bitnami <code>kubectl:1.30</code> image tag schema change in Velero's CRD upgrade hook). Root cause catalogued in apply log; documented as Known Limitation in submission email body.
Live metric flow into Mimir / Loki / Tempo	✗	

The deferred work is the in-cluster Prometheus agent install. The platform side — receive-end, dashboards, query plane — is the scaffolding the app team's instrumentation feeds into. Once the app exposes `/metrics` and the agent install lands, the dashboards populate.

CloudWatch tier (AWS-infra layer, live today)

CloudWatch evidence in this directory's `cloudwatch/` subdirectory + two console PNGs are the live-now slice. Captured via the reusable script `scripts/chaos/capture-cloudwatch-evidence.sh` (idempotent, re-runnable, `kubectl-or-AWS-CLI` fallback). Pairs naturally with the in-cluster log: anything that happens in the K8s API leaves an audit trail in CloudWatch; anything that hits AWS infra (EBS, DynamoDB, ELB) leaves a metric in CloudWatch's namespaces.

Reproducing this drill

Pre-req: a staging cluster matching the `helm/aegis-statefulset/` chart deployed (see `runbook docs/operations/runbooks/02-aws-bootstrap-and-chaos-demo.md`).

1. Run the chaos verifier (writes to `/tmp/`, exit-coded)
`scripts/chaos/demo-stateful-pod-kill.sh`

2. Capture the AWS-side corroboration for the same window

scripts/chaos/capture-cloudwatch-evidence.sh

```
# 3. Copy artefacts into docs/evidence/ for the submission record
cp /tmp/chaos-stateful-pod-kill-*.log docs/evidence/chaos-
stateful-pod-kill.log
# (CloudWatch JSON is already in docs/evidence/cloudwatch/ via the
capture script)
```

Manual artefacts (browser screenshots):

- Grafana stack landing → grafana-1-stack.png
 - Grafana dashboards list → grafana-2-dashboards-list.png
 - One dashboard opened with No data panels visible → grafana-3-service-availability-nodata.png
 - CloudWatch Logs Insights query for pod-delete audit events → cloudwatch-1-audit-pod-delete.png
 - CloudWatch Metrics graph of EBS IOPS over the chaos window → cloudwatch-2-ebs-volume-io.png
-

Cross-references

- **ADR-04** (backup, DR & HA) — the design these tests validate
- **ADR-01** (architecture & topology) — single-master-AZ + warm-standby pattern that makes pod-level recovery land on the same node
- **ADR-10 § 7** — the DR-posture cost dial that names the warm-vs- cold trade-off the cold-DR pick is judged against
- docs/operations/why-cold-dr.md — full reasoning chain for cold DR over active-passive
- docs/operations/region-failure-recovery.md — the runbook for the § 2 scenarios that this drill did not exercise